Executive Summary

In mathematics, there are thousands of complex calculations that are made to reach one result. Once students have learned the innerworkings of a mathematical methodology, they move on to being engineers, scientists, physicists, etc. However, these working professionals no longer need to complete complex operations to learn the technique of solving specific types of equations as they have already acquired these skills. MME is a Python-interpreter-based language specifically designed for natural language understanding in arithmetic, algebra, and basic calculus problems. Non-programmer individuals will now have a coding language that they have access to that does not have a complex learning curve with difficult syntax. MME will be a powerful tool in all industries requiring mathematics, and it will provide ease-of-use and efficiency.

White Paper: “Make Mathematics Easy (MME)”

Daisy Aptovska

Computer Science Program, The Pennsylvania State University at Abington

CMPSC 470: Compiler Construction

Professor Craig A. Nelson

March 1, 2026

Abstraction

Mathematics is often a challenging subject that takes many complex calculations. Software tools have developed around mathematics, and it is crucial that programming is accessible for all. While many programmers and students are well-versed in higher level mathematics, "Make Mathematics Easy" (MME hereafter) strives to provide a solution for non-programmers providing technical professionals outside of technology an easy methodology for complex calculations. MME will follow a basic natural-language-like structure that allows for non-programmers to solve accurate and efficient arithmetic, algebra, and basic calculus problems.

Introduction

In current mathematic software, we have access to powerful tools that perform instantaneous computations due to modern computing with tools such as MATLAB, other high-level programming languages, and even calculators. However, MME hopes to provide an efficient, word-based methodology for understanding mathematical equations, allowing uses to directly type phrases along with their equations to receive a step—by-step solved solution.

The Problem Statement

- **Accessibility:** Modern-day high-level programming languages still require prior knowledge of the languages syntax, making the power of modern-computing and programming less accessible to those who aren't technically versed. MME allows an AI-like accessibility to all individuals in need to efficient calculations with simple understanding and use of the MME programming language.

- **Interpretability:** Using a compiler is very efficient because of speed; however, there is lack of line-by-line debugging. Using an interpreter-based language as the intermediate code generation tool for MME provides interpretability where users can analyze bugs in their calculations line-by-line.

Background and Motivation

Over the last century, computers have had astounding breakthroughs, gained popularity, and have been integrated into daily life. For some background, high-level programming languages have gained popularity since the 1990s. They hold powerful programming abilities with ease-of-use for programmers with many built-in functionalities. Through the last few years, generative AI has gained immense international popularity, completely changing the workflow of both technical and non-technical working professionals, students, educators, and essentially everyone.

However, there are two key factors high-level programming languages and generative AI lack. High-level programming languages may have ease-of-use for skilled programmers, and newcomers still may require weeks to adapt to languages. Generative AI is still on the rise. It tends to make mistakes ("hallucinations"), and in high-risk situations such as healthcare or engineering, calculative mistakes are detrimental to the workflow.

With MME, the language will help eliminate these concerns. Users will quickly adapt to the natural-language-like syntax. Concerns for mistakes will be eliminated since MME will not use an AI-approach for calculations, but it will be backed with intermediate code generation by Python, an interpreter-based, high-level programming language that provides accurate calculations.

Core Language Features

The core features of MME include the following: basic arithmetic, algebraic expressions and equations, and simple calculus functions (derivatives and integrals). MME will use a natural-language-like syntax and a Python-based intermediate code generation to run these features. MME have efficiency from these features due to the interpreter-based nature. MME will allow users to type statements or upload .txt files (with specific formatting) with their MME code to run within the MME system (based in Python). The preview statements below of a .txt file demonstrate the syntax needed for .txt file MME code:

…

Solve for x: 2x + 1 = -y/3

Solve for y: 2x + 1 = -y/3

Derive: 2x^3 + 3x^2 + 5x + 1

…

The Type of System - Static Type System or Non-Static System

In CMPSC 470, this topic has not been discussed yet as of March 1, 2026.

Example of Syntax and Semantics

For syntax, MME will follow natural language based on how mathematical problems are commonly formatted. Using natural language for the syntax of MME assists with MMEs easy-of-use. The syntax will follow a structure as such: *keyword phrase operation + colon separation + equation/expression*. For example, the following statement would be syntactically valid is MME: “Solve: 3x = 5x + 3”. Another example may be the following: “Solve for y: x + 4 + y =

1”. Additionally, MME will be non-case sensitive on the keyword phrase operation, but it will be case-sensitive on the equation/expression since variables can vary. Lastly, MME will allow a single-line comment with “$” and a multi-line comment with “$...” to start and “…$” to end.

For semantics, a crucial feature that users need to focus on in MME is that their keyword operation is placed onto the right operation. Additionally, MME users will need to ensure they have the proper input of their equation/expression. For example, if an MME user wants to solve for x, MME automatically will solve for y if only the keyword “Solve” is used in the following example: “Solve: x + 1 = y + 5”. This statement is syntactically correct since MME knows to automatically solve for, but if the MME user’s goal was to solve for x, then the input is semantically incorrect.

Compiler Architecture

For the frontend of the MME interpreter, the structure will focus on lexical analysis, semantic analysis, and syntax analysis. For the backend of the MME interpreter, the code will be fed through to Python intermediate code generation to process MME text files.

Comparison to Existing Languages

To compare MME to other popular programming languages, this section focuses on how MME compares in easy-of-use, productivity, and performance. As for easy-of-use, MME’s core pillar is to provide easy-of-use with natural language phrases as the syntax of the language. MME will have keywords and key phrases to describe equations and their order such as “solve for x”, “integrate”, “derive”, etc. Through this simple methodology, MME is far easier to use than other high-level languages such as Python, Java, or C++ because in other high-level

languages, they require programmer-designed functions or classes to perform more complex multi-step operations.

As for productivity, MME hopes to provide productivity based on its ease-of-use. Users of MME will be able to quickly pick up the skills to use MME since the syntactical pattern of MME follows a natural language pattern. Users will not need to spend hours learning the syntax of the language, built-in functionalities, and practicing their coding abilities to utilize MME. While high-level languages like Python, Java, and C++ are powerful languages, they take several weeks to learn or even months for users who have never programmed before.

Lastly, for performance, MME will be as efficient as possible for an interpreter. While interpreters are slower by nature than compilers, the efficiency of Python is still quite powerful, and MME will use Python as its intermediate code generator. Java and C++ are more efficient in performance than Python as they follow a hybrid and full compiler-based structure, but this trade-off contributes to the ease-of-use of MME as users will have the power of line-by-line debugging.

Summary of the Benefits of the Language

MME holds many benefits for working professionals. Automating mathematics has been a long process since the beginning of computing, but simplifying this process for non-technical individuals has been difficult. Generative AI has been known to struggle with mathematics; therefore, efficient and accurate mathematical computing is not accessible for non-technical individuals. MME will provide easy-of-use, efficiency, and simplicity. Easy-of-use is essential for a programming language, so users can adapt quickly and understand the syntax and semantics of the language.

Vision for the Future of the Language

For the future of MME, the development of visualizations and even more complex calculations would be a great addition. Additionally, adding more keywords to MME will help develop the complexity of the natural language it can understand. Adding the functionality of sentence structure ("subject verb object" structure) can help with creating even more ease-of-use for the language. For MME supporting basic operations and understanding in its early development, in the future, it would be excellent to incorporate different subject areas of mathematics that support more complex operations and visualizations to assist working professionals.

References

Hierso, S. (2022). *Coding From 1849 to 2022: a Guide to The Timeline of Programming Languages.* IEEE Computer Society. https://www.computer.org/publications/tech-news/insider-membership-news/timeline-of-programming-languages

Imarchit19. (2025). *Difference Between Compiler and Interpreter*. GeeksforGeeks. https://www.geeksforgeeks.org/compiler-design/difference-between-compiler-and-interpreter/

Nelson, C. A. (n. d.) *Creating a White Paper.pdf.* Canvas.

Nelson, C. A. (n. d.) *Example of a Completed Programming Language Specification Document.pdf.* Canvas.

Nelson, C. A. (n. d.) *An Example of White Paper Proposal In Compiler Construction For A New Programming Language.pdf.* Canvas.

Nelson, C. A. (n. d.) *Language Specification Documentation Review.pptx.* Canvas.

Nelson, C. A. (n. d.) *Patricia Zero Specifications – Draft.pdf.* Canvas.

What are AI Hallucinations? (n.d.) IBM. https://www.ibm.com/think/topics/ai-hallucinations

Werner, J. (2024). *AI Is Usually Bad At Math. Here's Why It Matters.* Forbes. https://www.forbes.com/sites/johnwerner/2024/10/07/ai-is-usually-bad-at-math-heres-what-will-happen-if-it-gets-better/